AI Performance Hackathon

Team TOA

Autonomous vLLM Performance Tuning Agent

An AI agent that benchmarks, analyzes, and tunes vLLM inference servers without human intervention — safely, iteratively, and at scale.

Aanya Sharma
Agent Architecture

Rohan Awhad
Evaluation Harness

Thameem Ibrahim
Infrastructure

NVIDIA H100 80GB
GPU Target

Claude + GuideLLM
Powered By

The Problem

1. **Tuning kills the server.** Restarting vLLM inside the pod kills PID 1, which crashes the entire pod. The agent gets stuck in a recovery loop.
2. **No baseline comparison.** Without a stable reference, you can't tell if a change actually improved performance or if variance is fooling you.
3. **Manual tuning doesn't scale.** vLLM has 10+ tunable parameters with complex interactions. Testing combinations by hand is slow and error-prone.

```
# What used to happen:

agent → kill PID 1 to restart vLLM
pod   → CrashLoopBackOff
agent → retry... retry... retry...
agent → "Max iterations reached"

# No results. No comparison.
# Just a dead pod and wasted API tokens.
```

Our Solution: Pod-per-Experiment

1. **Baseline is read-only.** The original pod is never modified. It serves as a stable reference for every comparison.
2. **Fresh pod per experiment.** Each tuning attempt creates a new pod with different vLLM args, benchmarks it, then deletes it.
3. **Automated A/B testing.** Every experiment is compared against the same baseline with regression detection (2% threshold).
4. **Self-cleaning.** Experiment pods are deleted after benchmarking. `atexit` cleanup runs even on crashes.

```
# New architecture:

Baseline pod (read-only, never modified)
  • oc port-forward :8000
  • run_benchmark → baseline.json
  • fetch_vllm_logs → baseline config

Experiment pod (created per tuning attempt)
  • oc port-forward :8001, :8002, ...
  • run_benchmark → experiment.json
  • compare_benchmarks(baseline, experiment)
  • oc delete pod (cleanup)

# Real pod names from our run:
vllm-tune-1777144117  (chunked prefill)
vllm-tune-1777144304  (gpu-mem 0.95)
vllm-tune-1777144451  (gpu-mem + prefix)
```

Agent Workflow

Phase 1 — Baseline

1. Inspect GPU & vLLM config via `run_command`
2. Benchmark baseline with `run_benchmark`
3. Parse logs + extract metrics with `fetch_vllm_logs`

↓

Phase 2 — Experiments (loop)

4. Create experiment pod via `create_vllm_pod`
5. Benchmark experiment at new endpoint
6. Compare against baseline with `compare_benchmarks`
7. Delete pod & try next combination

↓

Phase 3 — Report

8. Call `done` with structured summary

Stopping Criteria

The agent keeps exploring until **10 consecutive experiments** show no improvement (>2% threshold). No premature stopping. Default max: 100 iterations.

13 Tools Available

`run_command` `read_file` `write_file` `run_benchmark` `fetch_vllm_logs` `read_benchmark_results` `compare_benchmarks` `analyze_trace` `map_kernel` `create_vllm_pod` `delete_vllm_pod` `check_preemptions` `done`

Claude Decides Everything

Which params to try, how to interpret results, when to combine winning configs, when failures are noise vs. real regressions, and when to stop.

System Prompt Engineering

Structured Workflow in the Prompt

```
TUNING WORKFLOW (follow this order strictly):
1. Benchmark the BASELINE pod (already running):
   a. Call run_benchmark with profile="balanced"
   b. Call fetch_vllm_logs (baseline pod logs)
   c. Call read_benchmark_results
   → Save the baseline JSON path for comparison.

2. For EACH tuning experiment:
   a. Call create_vllm_pod with vllm_args
      → Returns pod_name and endpoint
   b. Call run_benchmark with experiment endpoint
   c. Call fetch_vllm_logs with pod_name
   d. Call read_benchmark_results
   e. Call compare_benchmarks(baseline, experiment)
   f. Call delete_vllm_pod to clean up

3. NEVER kill processes on the baseline pod.
   NEVER restart the baseline pod.

STOPPING CRITERIA:
- Keep running until 10 CONSECUTIVE experiments
  show NO improvement over current best.
- Any experiment improving throughput OR latency
  by >2% resets the counter to zero.
- Do NOT stop early.
```

Tunable Parameters Taught to Claude

```
VLLM TUNABLE PARAMETERS:
 1. --max-num-seqs        (1-1024)
 2. --max-num-batched-tokens (256-32768)
 3. --gpu-memory-utilization (0.80-0.95)
 4. --enable-chunked-prefill (bool)
 5. --enable-prefix-caching  (bool)
 6. --max-model-len        (int)
 7. --enforce-eager        (bool)
 8. --tensor-parallel-size (1-8)
 9. --quantization   (fp8/awq/gptq)
10. --scheduling-policy (fcfs/priority)
```

Mandatory Post-Benchmark Steps

```
MANDATORY WORKFLOW FOR EACH BENCHMARK:
After EVERY run_benchmark call, you MUST:

1. CALL fetch_vllm_logs
   Parses 120+ regex patterns:
   server config, engine config, memory,
   compilation timing, warnings/errors

2. CALL read_benchmark_results
   Returns per-concurrency metrics:
   throughput, TTFT, ITL, TPOT (P50/P95/P99)
```

Engineering Deep Dive

PodManager

Creates pods from YAML template with modified vLLM args. Generates unique names (`vllm-tune-{ts}`), polls readiness every 5s, starts `oc port-forward` on incrementing ports (8001, 8002...). Tracks all procs for cleanup.

Auto Request Type

GuideLLM needs `--request-type`. We auto-detect: if model name contains "chat", "instruct", "it-" → `chat_completions`. Otherwise → `text_completions`. Fixed 400 errors on base models like opt-125m.

120+ Log Patterns

Regex parser extracts: vLLM version, non-default args, architecture, dtype, TP size, chunked prefill, attention backend, KV cache size, CUDA graph timing, torch.compile time, model memory, and all warnings/errors.

Directionality-Aware Comparison

compare_benchmarks knows throughput is higher=better, latency is lower=better. 2% threshold. Reports per-metric regressions and improvements across 45 metrics per concurrency level.

Crash-Safe Cleanup

`atexit.register(cleanup_all)` + try/finally in main loop. Even if the agent crashes mid-experiment, leftover pods are deleted. Port-forward processes are killed.

Prompt Caching

System prompt uses `cache_control: ephemeral`. On our 25-iteration run: 294K input tokens, but most were cache hits. Total cost: **$0.93** for the full tuning session.

Live Agent Output

Real output from our agent run on H100 — 25 iterations, 3 experiments, fully autonomous

```
>> Iteration 1/100
   Tool: run_command  {"command": "nvidia-smi"}
   NVIDIA H100 80GB HBM3  |  73807MiB / 81559MiB  |  CUDA 13.0
   VLLM::EngineCore  73798MiB

>> Iteration 2/100
   Tool: run_command  {"command": "cat /proc/1/cmdline | tr '\\0' ' '"}
   /usr/bin/python3 /usr/local/bin/vllm serve --model /models/facebook/opt-125m --port 8000 --dtype auto

>> Iteration 3/100
   Tool: run_benchmark  {"profile": "balanced"}
   Concurrency 1:  784.57 tok/s median  |  TTFT 31.5ms  |  ITL 1.2ms  |  100% success
   Concurrency 50: 11,738 tok/s median  |  TTFT 70.8ms  |  ITL 2.2ms  |  98.8% success
   → Saved: benchmark_results/balanced_20260425_150613.json

>> Iteration 9/100
   Tool: create_vllm_pod  {"vllm_args": ["--enable-chunked-prefill"]}
   Pod: vllm-tune-1777144117  Endpoint: http://localhost:8001

>> Iteration 13/100
   Tool: compare_benchmarks  baseline vs chunked-prefill
   Concurrency 50: 15 regressions, 6 improvements
   TTFT P95: 138ms → 156ms (+12.5%)  TTFT P99: 158ms → 175ms (+10.4%)
   Verdict: NOT beneficial for this workload

>> Iteration 15/100
   Tool: create_vllm_pod  {"vllm_args": ["--gpu-memory-utilization", "0.95"]}
   Pod: vllm-tune-1777144304  Endpoint: http://localhost:8002

>> Iteration 18/100
   Tool: compare_benchmarks  baseline vs gpu-mem-0.95
   Throughput: 11,739 → 12,254 tok/s (+4.4%)  100% success rate (vs 98.8%)

>> Iteration 20/100
   Tool: create_vllm_pod  {"vllm_args": ["--gpu-memory-utilization", "0.95", "--enable-prefix-caching"]}
   Pod: vllm-tune-1777144451  Endpoint: http://localhost:8003

>> Iteration 23/100
   Tool: compare_benchmarks  baseline vs gpu-mem-0.95+prefix-caching
   Throughput: 11,739 → 14,092 tok/s (+20.1%)
   TTFT P50:  70.8ms → 65.9ms (-6.9%)  ITL P50: 2.18ms → 1.98ms (-9.4%)
   Latency:   352ms → 317ms (-10.1%)

>> Iteration 25/100
   Tool: done  {"success": true}
```

Experiment Results (Concurrency 50)

Real H100 benchmark data — facebook/opt-125m, balanced profile (ISL=128, OSL=128)

CONFIGURATION	THROUGHPUT (TOK/S)	TTFT P50	ITL P50	TPOT P50	LATENCY P50	SUCCESS RATE	VERDICT
`Baseline (default args)`	11,739	70.8ms	2.18ms	2.75ms	352ms	98.8%	—
`--enable-chunked-prefill`	11,773 (+0.3%)	71.7ms **(+1.3%)**	2.11ms **(-3.4%)**	2.76ms (+0.4%)	354ms (+0.5%)	98.9%	**Mixed / Rejected**
`--gpu-memory-utilization 0.95`	12,254 **(+4.4%)**	69.3ms **(-2.1%)**	2.05ms **(-6.0%)**	2.57ms **(-6.3%)**	330ms **(-6.4%)**	**100%**	**Improved**
`--gpu-memory-utilization 0.95 --enable-prefix-caching`	**14,092 (+20.1%)**	**65.9ms (-6.9%)**	**1.98ms (-9.4%)**	**2.47ms (-10.0%)**	**317ms (-10.1%)**	99.1%	✅ **Best**

+20.1% Throughput Gain

-6.9% TTFT Reduction

-9.4% ITL Reduction

-10.1% Latency Reduction

3 Experiments Run

25 Agent Iterations

Cost & Efficiency

Claude API Usage (Actual Run)

294,686
Input Tokens

3,159
Output Tokens

26
API Calls

$0.93
Total Cost (Full Tuning Session)

Why So Cheap?

The system prompt (~5K tokens) is marked with `cache_control: ephemeral`. Across 26 API calls, it's only billed at full price once — subsequent calls use cached input. This gives us 90%+ cache hit rate on the prompt.

Agent Summary (Actual Output)

```
FINAL RECOMMENDATIONS:

1. Deploy with:
   --gpu-memory-utilization 0.95
   --enable-prefix-caching

2. Performance gains:
   20% throughput increase with improved latency

3. Why it works:
   Higher GPU memory utilization increases
   KV cache capacity, while prefix caching
   reduces redundant computation for
   repeated patterns

4. Production readiness:
   Excellent success rates (>99%) with
   substantial performance improvements

NEXT STEPS IF CONTINUING:
- Test --max-num-seqs for higher concurrency
- Experiment with --max-num-batched-tokens
- Consider tensor parallelism if multi-GPU
```

Thank You

Autonomous vLLM Performance Tuning Agent

+20.1%	13	120+	$0.93	0
Throughput Gain	Agent Tools	Log Parse Patterns	Total Agent Cost	Pods Harmed

Aanya Sharma
Agent Architecture

Rohan Awhad
Evaluation Harness

Thameem Ibrahim
Infrastructure

github.com/aas008/toa-vllm-hackathon